SECURITY AUDIT

Symbiotic

Euler / Pareto

FINAL REPORT

Aug 2026

BAILSEC.IO

Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Symbiotic – Euler / Pareto – Audit Report
Website	symbiotic.fi
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/symbioticfi/core-mirror/blob/80c346e07a47ec424d9ad7a3e7078ba5d521d6c5/src/contracts/
Resolution 1	https://github.com/symbioticfi/core-mirror/tree/87348bbecefddb1d7226a4a888848de464e37d1a/src/contracts
Resolution 2	https://github.com/symbioticfi/core-mirror/tree/a8846cdc650b67a0a22c773e21a46abe55245d50/src/contracts

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	2	1	1			
Low	8			8		
Informational	4			4		
Governance						
Total	14	1	1	12		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

ParetoOracle

The ParetoOracle contract prices a Pareto senior tranche token by reading the virtual price of that tranche from the Pareto IdleCDO and normalizing it to 1e18 precision. It extends the base Oracle, which enforces immutable minimum and maximum price bounds and reverts whenever the source price falls outside them, so this contract only supplies the raw price feed.

Invariants

- TOKEN_TO_REDEEM is always the AATranche of IDLE_CDO
- Prices returned by getPrice always lie within MIN_PRICE and MAX_PRICE as long as the vault is not defaulted.

Issue_01	ParetoOracle omits accrued but uncheckpointed management fees
Severity	**Low**
Description	ParetoOracle returns the CDO virtual price without accounting for management fees that have accrued since the last accounting update. The CDO deducts stored unpaid fees when calculating its managed value, but the fee for time elapsed since the previous checkpoint is only recorded during a state changing accounting operation. Until that operation occurs, ParetoOracle continues returning a price that excludes the accrued liability. A subsequent withdrawal request or accounting update records the fee and reduces the economic value of the tranche. Before that update, LiquidLane can purchase the tranche at an overstated price, and the vault can issue or redeem shares using an overstated account value. A shareholder withdrawing before the checkpoint can transfer part of the accrued fee burden to the remaining shareholders.
Recommendations	Calculate the oracle price using a preview that includes management fees accrued through the current block. Alternatively, checkpoint the CDO before swaps and vault operations that depend on the Pareto price.
Comments / Resolution	Acknowledged.

ParetoAccount

The ParetoAccount contract is the account implementation that holds Pareto tranche tokens on behalf of a vault and drives redemptions through Pareto's epoch-based withdrawal queue. Pareto redemptions are asynchronous: a withdrawal request is registered against a future epoch, and only once that epoch is priced and its pending claims are cleared can the underlying be claimed. The contract tracks the epochs it has open, values them for the vault, and settles any asset mismatch through the inherited CoW converter.

Invariants

- queueEpochs contains no duplicate epoch numbers
- Total assets reflect the sum of unclaimed queue positions plus held redemption tokens, expressed in vault assets

Issue_02	Funds can be stranded in case of vault closure
Severity	**Medium**
Description	In _sync, Pareto withdrawals are only carried out when the epoch is running. This is because the Queue contract only allows withdrawals during epochs. The issue is that if the vault suffers a loss after which it is deemed to be retired, there is no way for the account to redeem their last remaining funds anymore if the epochs are not running. During the buffer times, Pareto allows redemptions by directly interacting with the IdleCDOEpochVariant contract and calling requestWithdraw. However, the Account contract here does not support this functionality, so any Pareto vault tokens here will stay stranded in this contract permanently.
Recommendations	Consider implementing an escape hatch for such scenarios, and having the contract call requestWithdraw on the IdleCDOEpochVariant contract directly.
Comments / Resolution	Fixed: ParetoAccount now uses the direct CDO withdrawal path when no epoch is running and accounts for the resulting receipt tokens.

Issue_03	Pareto default can leave AA_FalconX priced after withdrawals stop
Severity	**Medium**
Description	ParetoOracle prices AA_FalconX only from the Pareto CDO virtual price and does not check whether the CDO has defaulted. If the borrower defaults, the Pareto CDO marks itself as defaulted and stops normal withdrawal epochs. ParetoAccount then skips new withdrawal requests because isEpochRunning() is false. However, the account and LiquidLaneAdapter can still value held or newly received AA_FalconX through ParetoOracle. The CDO virtual price can still include strategy assets at their previous value after default. Therefore, AA_FalconX can remain valued and accepted as if it were redeemable at that price, even though withdrawals are stopped and the underlying value may no longer be recoverable.
Recommendations	Check the CDO default state in the oracle or account. When defaulted, pause AA_FalconX intake and use a dedicated recovery or migration flow.
Comments / Resolution	Partially fixed with potential risk. Defaulted AA_FalconX is valued at zero, but intake remains enabled, and the zero valuation can allow an account holding recoverable tokens to be removed from normal accounting and sweeping.

Issue_04	Skipped Pareto withdrawal epochs cannot be recovered by the Account
Severity	**Low**
Description	sync submits held tranche tokens for the next strategy epoch and stores that epoch in queueEpochs. The Idle withdrawal queue later processes requests only for the strategy's current epoch. processWithdrawRequests does not accept an epoch argument. If an operator advances the strategy without processing a requested epoch, the skipped request becomes unreachable through the normal processing function. Later calls process the newer current epoch and cannot return to the skipped one. The old request remains recorded with a zero withdrawal price, so the Account continues retaining it in queueEpochs. Idle provides deleteWithdrawRequest as the recovery path for an unprocessed request. That function reads the request using msg.sender and returns the tranche tokens to the same address. In this integration, the requester is the Account proxy. External users cannot cancel the request on its behalf, and the Account does not expose a function that forwards the cancellation call. As a result, the affected principal remains in the Idle queue and cannot complete the normal redemption flow. The Account continues including the request in totalAssets using the live oracle price even though the funds are unavailable to the vault. Repeated skipped epochs also increase the size of queueEpochs and make every valuation and synchronization call more expensive. Creating the condition requires an authorized operator to miss processing and then advance the strategy lifecycle. This lowers the likelihood, but there is no recovery through the current Account, LiquidLane, Delegator, or Vault interfaces once it happens. Recovering the position requires an Account implementation upgrade.
Recommendations	Add an Account function that calls deleteWithdrawRequest for a skipped epoch from the Account proxy.
Comments / Resolution	Acknowledged.

Issue_05	queueEpochs array can grow unbounded for unclaimable withdrawals
Severity	**Low**
Description	In _sync, the queueEpochs array is iterated over. An element is added to queueEpochs whenever there is any idle balance in the contract, ready for withdraw requests. The issue is that requestWithdraw can be called and queueEpochs array can be extended even for a simple 1 wei withdrawal. If the price of the vault is lower than 1e6 and this 1 wei is the only redemption for that epoch, this can result in a redemption fulfilment of 0 amount. In such a case, processWithdrawalClaims in IdleCDOEpochQueue fails with Is0, so the redemption can never be completed. Since the redemption can never be completed, the queueEpochs array always contains that epoch and cannot be removed. If this happens over multiple epochs, this array can keep growing unbounded.
Recommendations	Consider adding a minimum redemption amount gate in _sync.
Comments / Resolution	Acknowledged.

Issue_06	_totalAssets can change sharply, allowing sandwich attacks/frontrunning
Severity	Low
Description	In _totalAssets, the unprocessed tokens are priced according to the current oracle value, and the processed tokens at the recorded price. The issue is that in case of any loss, the prices will change within a single block. The oracle price can be updated, affecting unprocessed fund valuation, and the final settlement value can be different from the calculated redemptionTokenAmount value, since processWithdrawalClaims can update the price. Users can frontrun such price decrements with a withdrawal to safeguard their funds against losses, and force the losses onto other vault participants.
Recommendations	Consider acknowledging this issue, or adding a timelock for the vault.
Comments / Resolution	Acknowledged.

Issue_07	Ready Pareto claims can be rolled back by a failing new withdrawal
Severity	**Low**
Description	ParetoAccount.sync() first claims USDC from completed Pareto withdrawal epochs. It then submits any held AA_FalconX balance into a new withdrawal request. If the new requestWithdraw() call reverts, the complete sync() transaction reverts, including the earlier successful claims. For example, the Pareto queue checks that the requesting wallet remains allowed, but ParetoAccount does not check this before calling requestWithdraw(). As a result, ready USDC claims remain in the withdrawal queue until the account becomes allowed again or the failing new withdrawal path is removed.
Recommendations	Check whether the account is allowed before creating a new withdrawal request.
Comments / Resolution	Acknowledged.

Issue_08	Pareto redemption proceeds assume a 1:1 rate with the vault asset
Severity	**Low**
Description	ParetoAccount supports vault assets that differ from Pareto's redemption token, such as a USDT vault receiving USDC. However, _redemptionTokenToAssets only adjusts token decimals and assumes both assets have equal value. If the tokens depeg, totalAssets misprices the redeemed balance until it is swapped, affecting vault accounting and limits.
Recommendations	Require the vault asset and redemption token to match, or use an exchange-rate oracle for the conversion.
Comments / Resolution	Acknowledged.

Issue_09	ParetoAccount cannot cancel pending withdrawal requests
Severity	**Informational**
Description	ParetoAccount submits AA_FalconX withdrawal requests through the Pareto withdrawal queue. The queue provides deleteWithdrawRequest(epoch) to cancel an unprocessed request, but only the original requester can call it. Here, the requester is ParetoAccount itself. However, ParetoAccount does not expose a function that calls deleteWithdrawRequest(). Therefore, if an epoch is delayed or the account needs to cancel an unprocessed request, there is no normal account flow to recover the queued AA_FalconX before the queue processes it.
Recommendations	Consider adding an owner-gated function to cancel an unprocessed withdrawal request or document the intended recovery process.
Comments / Resolution	Acknowledged.

Issue_10	Processed Pareto withdrawals use a provisional settlement price
Severity	Informational
Description	ParetoAccount uses epochWithdrawPrice to value a processed withdrawal as soon as this price becomes available. However, the withdrawal is not claimable while epochPendingClaims(epoch) is non-zero. When Pareto later settles the pending claims, it can update epochWithdrawPrice if the actual received USDC differs from the earlier amount. Therefore, totalAssets() can change between withdrawal processing and final settlement.
Recommendations	Document that processed Pareto withdrawals can be repriced until their pending claims are settled.
Comments / Resolution	Acknowledged.

EulerAdapter

The EulerAdapter contract is a VaultV2 adapter that routes vault assets into a single Euler Lend ERC4626 vault and back out again. It holds the Lend vault shares itself and tracks them in totalShares rather than relying on the share balance, so reward tokens or donations do not distort accounting. Alongside the base Adapter it inherits CoWSwapConverter for swapping incidental tokens into the vault asset and MerklClaimer for collecting Merkl reward distributions.

Privileged Functions

- allocate
- deallocate
- requestDeallocate

Invariants

- totalShares equals the Lend vault shares minted to the adapter minus those redeemed by it
- Conversions never consume the Lend vault share token or the vault asset as input, and always produce the vault asset

Issue_11	Claimed Merkl rewards are excluded from EulerAdapter totalAssets
Severity	**Low**
Description	EulerAdapter can claim Merkl reward tokens, but totalAssets() only counts the adapter's vault-asset balance and its Euler Lend vault shares. If a claimed Merkl reward is not the vault asset, the reward token is not included in totalAssets() until it is converted into the vault asset. The adapter can therefore temporarily report a lower NAV than the value it holds. Users may deposit while the rewards are excluded and receive shares too cheaply. They can then benefit when the rewards are later converted and included in the vault asset balance.
Recommendations	Convert claimed reward tokens promptly, or account for their value before allowing them to affect vault share pricing.
Comments / Resolution	Acknowledged.

Issue_12	allocatable is not implemented
Severity	**Low**
Description	During allocation, the Delegator tries to allocate the full amount available to the adapter, only gated by the limitOf of the adapter and its allocatable amount. However, allocatable is never overridden in this contract and always reports type(uint256).max. So the Delegator always tries to allocate the entire balance amount or the entire limit amount possible, irrespective of how much liquidity the vault can take, gated by its internal maxMintInternal parameter. This leads to unexpected reverts and ineffective utilization of the adapter cap, since the Delegator can try to delegate funds to it to meet the limit cap, but revert due to the maxMintInternal cap.
Recommendations	Consider overriding the allocatable function to accurately return the actual available cap amount.
Comments / Resolution	Acknowledged.

Issue_13	Duplicate base asset lookup in convert
Severity	**Informational**
Description	In EulerAdapter.convert, the vault base asset is fetched twice via IERC4626(vault).asset() — once when validating tokenIn and again when validating tokenOut. Both checks need the same value, but each call is a separate external view into the parent vault. Each call costs an external invocation plus a read of the vault's fixed asset address.
Recommendations	Cache the base asset in a local variable at the start of convert, then reuse it for both token checks.
Comments / Resolution	Acknowledged.

Issue_14	Euler adapter never opts into balance tracker rewards
Severity	**Informational**
Description	EulerAdapter deposits into an Euler Lend vault and holds shares as address(this). Euler vaults can use a balance tracker to distribute incentives based on share balances, but each account must call enableBalanceForwarder() to opt in. The adapter never does this. It only uses standard ERC4626 deposit and withdraw. MerklClaimer handles Merkl claims separately and does not register the adapter with Euler's on-chain tracker. This only matters if the lend vault has a balance tracker configured. If it does, the adapter's shares are excluded from reward accounting while it may hold a large deposit. Symbiotic vault depositors miss that yield.
Recommendations	Call enableBalanceForwarder() on the lend vault during init or first deposit when balanceTrackerAddress() is non-zero.
Comments / Resolution	Acknowledged.